透明・ハッシュベース ZK 証明系の見取り図 Toy computation で見る Ligero, zk-STARK, Aurora, Virgo

概要

本稿は、透明でハッシュ関数ベースのゼロ知識証明系としてよく言及される Ligero, zk-STARK, Aurora, Virgo を、同じ toy computation

$$y = (x+1)(x+2)(x+3)$$

を使って比較するためのノートである。狙いは、各方式が「同じ計算」を何に変換して証明しているのかを、学部生にも追える形で並べることである。特に後半では、Aurora の *rowcheck* と Virgo の *sumcheck* を式レベルで比較し、「どこが本質的に似ていて、どこが違うか」を明確にする。

本稿は理解のための再構成であり、実装仕様書ではない。そのため、小さな有限体 $\mathbb{F}_{17}$ と小さな評価領域を使い、実プロトコルが使う FFT-friendly な大きな領域、LDE、Merkle コミット、Fiat–Shamir、blinding、zkVPD の細部は必要な範囲でのみ触れる。ただし、数式の骨格は原論文の枠組みに従っている [1, 2, 3, 4, 5, 6]。

目次

1 導入 2

2 Toy computation 3
- 2.1 問題設定 . . . 3
- 2.2 回路として見たときの局所関係 . . . 3

3 Ligero ではどう見えるか 3

4 zk-STARK ではどう見えるか 4
- 4.1 実行トレース . . . 4
- 4.2 AIR 制約 . . . 5
- 4.3 トレース多項式 . . . 5
- 4.4 遷移制約の分母払い . . . 5

4.5 合成多項式の形 6

5 Aurora ではどう見えるか 7
5.1 R1CS への落とし込み 7
5.2 具体的な A, B, C 行列 7
5.3 rowcheck の本質 8
5.4 toy としての明示的な rowcheck 多項式 8

6 Virgo ではどう見えるか 9
6.1 layered circuit 9
6.2 multilinear extension 9
6.3 layer $1 \to 2$ の wiring selector 10
6.4 plain GKR / Virgo の sumcheck 恒等式 10
6.5 sumcheck の第一ラウンド多項式 11
6.6 Virgo の ZK 化（toy レベルの模式式） 11

7 Aurora の rowcheck と Virgo の sumcheck の式レベル比較 12
7.1 まず何が「同じ」なのか 12
7.2 式を真横に置く 12
7.3 どこが本質的に違うのか 13
7.4 Virgo の一層を Aurora 風に flatten すると何が見えるか 14
7.5 比較表 15

8 全体の見取り図 15

9 結論 15

1 導入

これら四つの方式はすべて「transparent（trusted setup 不要）」という大きな共通点を持つが、何を主役のデータ構造にするかが大きく違う。ざっくり言うと、Ligero は witness の**行列**、zk-STARK は計算の**実行トレース**、Aurora は **R1CS**、Virgo は **layered arithmetic circuit** の **各層の値**を主役にする [1, 3, 4, 5]。

この違いが、そのまま

- どの多項式を作るか、
- verifier が何を spot-check するか、
- 計算を「平坦化」して見るか、「層」を保ったまま見るか

の違いにつながる。本稿では一つの toy を使って、この差をできるだけ手で追える形で書き下す。

2 Toy computation

2.1 問題設定

有限体 $\mathbb{F}_{17}$ 上で、公開値 y に対して

$$y = (x+1)(x+2)(x+3) \tag{1}$$

を満たす秘密 x を知っていることを証明したい。

具体例として

$$x = 2$$

を witness とすると、

$$u_1 = x+1 = 3, \qquad u_2 = x+2 = 4, \qquad u_3 = x+3 = 5,$$
$$m = u_1 u_2 = 12, \qquad y = m u_3 = 12 \cdot 5 = 60 \equiv 9 \pmod{17}.$$

したがって、公開主張は

$$\exists x \in \mathbb{F}_{17} \quad \text{s.t.} \quad (x+1)(x+2)(x+3) = 9$$

である。

2.2 回路として見たときの局所関係

この計算は、以下の五つの局所関係に分解できる。

$$u_1 = x+1, \tag{2}$$
$$u_2 = x+2, \tag{3}$$
$$u_3 = x+3, \tag{4}$$
$$m = u_1 u_2, \tag{5}$$
$$y = m u_3. \tag{6}$$

この同じ意味の計算を、Ligero は「表」、zk-STARK は「実行ログ」、Aurora は「R1CS」、Virgo は「層ごとの値」として眺める。

3 Ligero ではどう見えるか

Ligero は、算術回路の witness をほぼ正方形の行列に並べ、各行を Reed–Solomon 符号でエンコードし、ランダムな行線形結合と 少数のランダム列を見ることで整合性を検査する。大づかみに言えば、「巨大な表全体を送る代わりに、符号語化した表を spot-check する」方式である [1]。

この toy では、たとえば

$$w = (1, x, u_1, u_2, u_3, m, y, 0) = (1, 2, 3, 4, 5, 12, 9, 0)$$

を二行四列に並べて

$$U_{\mathrm{raw}} = \begin{pmatrix} 1 & x & u_1 & u_2 \\ u_3 & m & y & 0 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 12 & 9 & 0 \end{pmatrix} \tag{7}$$

と書ける。実際の Ligero ではこの生行列をそのまま使うのでなく、各行を低次数多項式の評価列として冗長化する。

この toy の制約は、(2)–(4) が線形制約、(5)–(6) が二次制約である。後者は

$$r = (u_1, m), \qquad s = (u_2, u_3), \qquad t = (m, y)$$

と置くと

$$r \circ s = t$$

という成分ごとの積関係にまとめられる。Ligero の感覚では、「witness の表」と「その表の上で成り立つ線形・二次の局所関係」が出発点になる [1]。

■直感　Ligero の verifier は表全体を読まない。ランダム係数で行を混ぜたときに嘘が消える確率は低く、さらにランダム列を開かせれば、行方向の混合結果と列の中身の間に矛盾が出やすい。このため、$\tilde{O}(\sqrt{s})$ 程度の通信で大きな表の整合性をかなり強く検査できる [1]。

4　zk-STARK ではどう見えるか

4.1　実行トレース

zk-STARK 系では、同じ計算を「回路」よりも **実行トレース**として書くのが自然である。本 toy では、以下の三ステップのプログラムとして書ける。

- 初期状態: $i = 0,\ acc = 1$
- 三回繰り返す:

$$i \leftarrow i + 1, \qquad acc \leftarrow acc \cdot (x + i)$$

witness $x = 2$ ではトレース表は

row	x	i	acc
0	2	0	1
1	2	1	3
2	2	2	12
3	2	3	9

となる。

STARK ではこのトレースを AIR (Algebraic Intermediate Representation) に落とし、各列が低次数多項式から来ていることを FRI で検査する [3, 2]。

4.2 AIR 制約

隣接行の関係は

$$x' - x = 0, \tag{8}$$
$$i' - i - 1 = 0, \tag{9}$$
$$acc' - acc \cdot (x + i + 1) = 0. \tag{10}$$

また境界条件として

$$i_0 = 0, \qquad acc_0 = 1, \qquad i_{\mathrm{last}} = 3, \qquad acc_{\mathrm{last}} = 9 \tag{11}$$

を課せばよい。

4.3 トレース多項式

$\mathbb{F}_{17}^{\times}$ には位数 4 の部分群があり、

$$\omega = 4, \qquad \omega^4 = 1$$

なので

$$H = \{1, \omega, \omega^2, \omega^3\} = \{1, 4, 16, 13\}$$

をトレースドメインとして取れる。

この四点上で各列を補間すると

$$x(X) = 2, \tag{12}$$
$$i(X) = 10 + 10X + 8X^2 + 6X^3, \tag{13}$$
$$a(X) := acc(X) = 2 + 16X + 13X^2 + 4X^3. \tag{14}$$

たとえば

$$i(1) = 0, \quad i(4) = 1, \quad i(16) = 2, \quad i(13) = 3$$

であり、たしかにトレース列と一致する。

4.4 遷移制約の分母払い

遷移を課すのは最後の行以外なので、

$$H_{\mathrm{tr}} = \{1, 4, 16\}$$

と置き、その vanishing polynomial を

$$Z_{\mathrm{tr}}(X) = (X-1)(X-4)(X-16) = X^3 + 13X^2 + 16X + 4 \tag{15}$$

とする。

すると (8)–(10) に対応して

$$N_x(X) = x(\omega X) - x(X), \tag{16}$$
$$N_i(X) = i(\omega X) - i(X) - 1, \tag{17}$$
$$N_a(X) = a(\omega X) - a(X)\big(x(X) + i(X) + 1\big) \tag{18}$$

を定義できる。この toy では計算すると

$$N_x(X) = 0, \tag{19}$$
$$N_i(X) = 4\, Z_{\mathrm{tr}}(X), \tag{20}$$
$$N_a(X) = Z_{\mathrm{tr}}(X)\,(10X^3 + 15X^2 + 11) \tag{21}$$

となり、たしかに遷移制約が H_{tr} 上で成り立っていることが分かる。

境界条件 (11) も同様に、

$$\frac{i(X)}{X-1}, \qquad \frac{a(X)-1}{X-1}, \qquad \frac{i(X)-3}{X-13}, \qquad \frac{a(X)-9}{X-13}$$

が多項式であることとして書ける。実際、

$$\frac{i(X)}{X-1} = 6X^2 + 14X + 7, \tag{22}$$
$$\frac{a(X)-1}{X-1} = 4X^2 + 16, \tag{23}$$
$$\frac{i(X)-3}{X-13} = 6X^2 + X + 6, \tag{24}$$
$$\frac{a(X)-9}{X-13} = 4X^2 + 14X + 11. \tag{25}$$

4.5 合成多項式の形

STARK では最終的に、ランダム係数 α_j, β_j を使って

$$\begin{aligned} C(X) = {} & \alpha_1 \frac{N_x(X)}{Z_{\mathrm{tr}}(X)} + \alpha_2 \frac{N_i(X)}{Z_{\mathrm{tr}}(X)} + \alpha_3 \frac{N_a(X)}{Z_{\mathrm{tr}}(X)} \\ & + \beta_1 \frac{i(X)}{X-1} + \beta_2 \frac{a(X)-1}{X-1} + \beta_3 \frac{i(X)-3}{X-13} + \beta_4 \frac{a(X)-9}{X-13} \end{aligned} \tag{26}$$

のような **composition polynomial** を作り、これが所定の低次数以内であることを FRI で検査する [3, 2]。

ここで重要なのは、STARK の心が

$$\text{実行トレース} \longrightarrow \text{列多項式} \longrightarrow \text{遷移と境界の合成多項式}$$

という流れにある点である。

5 Aurora ではどう見えるか

5.1 R1CS への落とし込み

Aurora は R1CS relation を出発点にし、そのうえで **lincheck** と **rowcheck** を組み合わせて transparent な IOP を作る。R1CS は

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle \qquad (i = 1, \ldots, m) \tag{27}$$

の形の制約系である [4]。

本 toy では

$$z = (1, y, x, u_1, u_2, u_3, m) = (1, 9, 2, 3, 4, 5, 12)$$

と置けばよい。

5.2 具体的な A, B, C 行列

列順を

$$(1,\ y,\ x,\ u_1,\ u_2,\ u_3,\ m)$$

と固定すると、五本の制約

$$(x+1)\cdot 1 = u_1, \quad (x+2)\cdot 1 = u_2, \quad (x+3)\cdot 1 = u_3, \quad u_1 u_2 = m, \quad m u_3 = y$$

は

$$A = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 2 & 0 & 1 & 0 & 0 & 0 & 0 \\ 3 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \tag{28}$$

$$C = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{29}$$

で与えられる。

したがって

$$a := Az = \begin{pmatrix} 3 \\ 4 \\ 5 \\ 3 \\ 12 \end{pmatrix}, \qquad b := Bz = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 4 \\ 5 \end{pmatrix}, \qquad c := Cz = \begin{pmatrix} 3 \\ 4 \\ 5 \\ 12 \\ 9 \end{pmatrix}. \tag{30}$$

そして成分ごとに

$$a \circ b = c$$

が成り立つ。

5.3 rowcheck の本質

Aurora の rowcheck は、まさに

$$a \circ b = c \tag{31}$$

を低次数多項式の世界に押し込む部品である。一方、どの a, b, c が本当に Az, Bz, Cz から来たかは lincheck が担当する。したがって Aurora 側の役割分担は

$$\text{lincheck：線形写像の正しさ} \qquad \text{と} \qquad \text{rowcheck：成分ごとの積}$$

である [4]。

5.4 toy としての明示的な rowcheck 多項式

説明のため、row domain を

$$H_1 = \{0, 1, 2, 3, 4\} \subset \mathbb{F}_{17}$$

と取って (30) を補間する。このとき

$$f_a(X) = 8X^3 + 10X^2 + 3, \tag{32}$$
$$f_b(X) = 11X^4 + 11X^3 + 9X^2 + 3X + 1, \tag{33}$$
$$f_c(X) = 9X^4 + 15X^3 + 11X^2 + 3 \tag{34}$$

となる。また

$$Z_{H_1}(X) = \prod_{i=0}^{4}(X - i) = X^5 + 7X^4 + X^3 + X^2 + 7X. \tag{35}$$

計算すると

$$f_a(X)f_b(X) - f_c(X) = Z_{H_1}(X)\left(3X^2 + 7X + 11\right). \tag{36}$$

したがって、この toy における rowcheck の商多項式は

$$q_{\mathrm{row}}(X) = 3X^2 + 7X + 11. \tag{37}$$

■重要な注意　実際の Aurora は FFT-friendly な領域、パディング、LDE、RS コード語を使う。ここで $H_1 = \{0, 1, 2, 3, 4\}$ を選んでいるのは、rowcheck が

$$f_a f_b - f_c \text{ が row-domain 上で消える} \iff Z_{H_1} \text{ で割れる}$$

という形で働くことを、最小の toy で目に見える形にするためである [4]。

6　Virgo ではどう見えるか

6.1　layered circuit

Virgo/GKR 的には、回路は層ごとのゲート値で見るのが自然である。厳密に言えば、本 toy で u_1, u_2, u_3 を作るにはさらに一段下に加算層が必要だが、ここでは掛け算部分を比較の中心にしたいので、すでに

$$u_1 = 3, \qquad u_2 = 4, \qquad u_3 = 5, \qquad 1$$

が得られているところから始める。

そこで

$$V_2 = [u_1, u_2, u_3, 1] = [3, 4, 5, 1], \qquad V_1 = [u_1 u_2,\ u_3 \cdot 1] = [12, 5], \qquad V_0 = [9] \tag{38}$$

と置く。

6.2　multilinear extension

Layer 2 は $\{0, 1\}^2$ 上の関数として

$$V_2(0, 0) = 3, \quad V_2(0, 1) = 4, \quad V_2(1, 0) = 5, \quad V_2(1, 1) = 1$$

と書ける。その multilinear extension は

$$\widetilde{V}_2(X_1, X_2) = 3 + 2X_1 + X_2 + 12X_1X_2. \tag{39}$$

Layer 1 は $\{0, 1\}$ 上の関数

$$V_1(0) = 12, \qquad V_1(1) = 5$$

なので

$$\widetilde{V}_1(T) = 12(1 - T) + 5T = 12 + 10T. \tag{40}$$

Layer 0 は定数なので

$$\widetilde{V}_0 = 9. \tag{41}$$

6.3 layer $1 \to 2$ の wiring selector

Layer 1 の二つの multiplication gate は

- gate 0: child は $(00, 01)$
- gate 1: child は $(10, 11)$

に対応する。これを示すため、$\boldsymbol{x} = (x_1, x_2)$, $\boldsymbol{y} = (y_1, y_2)$ と書いて

$$M_0(\boldsymbol{x}, \boldsymbol{y}) = (1 - x_1)(1 - x_2)(1 - y_1)y_2, \tag{42}$$
$$M_1(\boldsymbol{x}, \boldsymbol{y}) = x_1(1 - x_2)y_1 y_2 \tag{43}$$

を定義する。これらはそれぞれ $(\boldsymbol{x}, \boldsymbol{y}) = (00, 01)$ と $(10, 11)$ を拾う indicator の multilinear extension である。

親ゲートのラベル $g \in \mathbb{F}_{17}$ を用いれば、layer 1 の wiring predicate は

$$\widetilde{\mathrm{mul}}_2(g, \boldsymbol{x}, \boldsymbol{y}) = (1 - g)M_0(\boldsymbol{x}, \boldsymbol{y}) + gM_1(\boldsymbol{x}, \boldsymbol{y}) \tag{44}$$

と書ける。

6.4 plain GKR / Virgo の sumcheck 恒等式

この toy の layer $1 \to 2$ には加算ゲートがないので、GKR の層関係式は

$$\widetilde{V}_1(g) = \sum_{\boldsymbol{x}, \boldsymbol{y} \in \{0,1\}^2} \widetilde{\mathrm{mul}}_2(g, \boldsymbol{x}, \boldsymbol{y})\, \widetilde{V}_2(\boldsymbol{x})\widetilde{V}_2(\boldsymbol{y}) \tag{45}$$

に簡約される。

右辺の被積分関数（実際には被加算関数）を

$$G_g(\boldsymbol{x}, \boldsymbol{y}) = \widetilde{\mathrm{mul}}_2(g, \boldsymbol{x}, \boldsymbol{y})\, \widetilde{V}_2(\boldsymbol{x})\widetilde{V}_2(\boldsymbol{y}) \tag{46}$$

と置くと、(45) は

$$\widetilde{V}_1(g) = \sum_{\boldsymbol{x}, \boldsymbol{y} \in \{0,1\}^2} G_g(\boldsymbol{x}, \boldsymbol{y}) \tag{47}$$

である。

実際、

$$\sum_{\boldsymbol{x}, \boldsymbol{y}} G_g(\boldsymbol{x}, \boldsymbol{y}) = (1 - g)\widetilde{V}_2(00)\widetilde{V}_2(01) + g\widetilde{V}_2(10)\widetilde{V}_2(11)$$

だから

$$= (1 - g) \cdot 3 \cdot 4 + g \cdot 5 \cdot 1 = 12 + 10g = \widetilde{V}_1(g),$$

と確かめられる。

6.5 sumcheck の第一ラウンド多項式

変数順を

$$x_1,\ x_2,\ y_1,\ y_2$$

とすると、sumcheck の第一ラウンドで prover が送る partial-sum 多項式は

$$q_1(T) = \sum_{x_2,y_1,y_2\in\{0,1\}} G_g(T, x_2, y_1, y_2). \tag{48}$$

この toy では計算すると

$$q_1(T) = (9+10g)T^2 + (13+7g)T + (12+5g). \tag{49}$$

そして

$$q_1(0) + q_1(1) = 12 + 10g = \widetilde{V}_1(g)$$

が成り立つ。これが sumcheck の最初の検査条件である [6, 5]。

6.6 Virgo の ZK 化（toy レベルの模式式）

Virgo では plain GKR をそのまま使うのでなく、各層の $\widetilde{V}_i$ をマスクした $\dot{V}_i$ に持ち上げ、さらに sumcheck transcript 自体を隠すための δ_i を加える [5]。

toy レベルの模式式として、

$$\dot{V}_1(t) = \widetilde{V}_1(t) + t(1-t)\sum_{b\in\{0,1\}} R_1(t,b)$$

と置き、現在の claim 点を $u, v \in \mathbb{F}_{17}$、ランダム係数を α, β, γ とすると、layer $1 \to 2$ に対する ZK 版の被加算多項式は

$$\begin{aligned}
F_1^{(\mathrm{ZK})}(x_1,x_2,y_1,y_2,b) = (1-b)\Big(&(\alpha(1-u)+\beta(1-v))M_0(\boldsymbol{x},\boldsymbol{y}) \\
&+ (\alpha u+\beta v)M_1(\boldsymbol{x},\boldsymbol{y})\Big)\dot{V}_2(\boldsymbol{x})\dot{V}_2(\boldsymbol{y}) \\
&+ (1-x_1)(1-x_2)(1-y_1)(1-y_2) \\
&\cdot\Big(\alpha u(1-u)R_1(u,b) + \beta v(1-v)R_1(v,b)\Big) \\
&+ \gamma\,\delta_1(x_1,x_2,y_1,y_2,b).
\end{aligned} \tag{50}$$

この形は Virgo の ZK 化の骨格、すなわち

- plain GKR 部分
- $\widetilde{V}_i$ から $\dot{V}_i$ への補正
- transcript マスク

を、一つの toy layer に落としたものである [5]。

7 Aurora の rowcheck と Virgo の sumcheck の式レベル比較

7.1 まず何が「同じ」なのか

両者が似て見える理由ははっきりしている。どちらも

- 多数の局所的な乗法関係を
- 低次数多項式に圧縮し
- verifier は少数のランダム点だけを見る

という設計を取っているからである。Aurora ではそれが row-wise な Hadamard 積であり、Virgo では layer-wise な wiring selector 付き hypercube 和である [4, 5]。

7.2 式を真横に置く

■Aurora rowcheck　Aurora では

$$a = Az, \qquad b = Bz, \qquad c = Cz$$

を用いて

$$a \circ b = c$$

を示したい。toy ではそれを補間して

$$f_a(X) f_b(X) - f_c(X) = Z_{H_1}(X)\, q_{\mathrm{row}}(X)$$

と書けた。ここで

$$q_{\mathrm{row}}(X) = 3X^2 + 7X + 11$$

である。つまり Aurora rowcheck の主張は

$$\forall \alpha \in H_1, \quad f_a(\alpha) f_b(\alpha) = f_c(\alpha) \quad \Longleftrightarrow \quad Z_{H_1} \mid (f_a f_b - f_c) \tag{51}$$

である。

■Virgo sumcheck　Virgo では同じ乗法関係を

$$\widetilde{V}_1(g) = \sum_{\boldsymbol{x},\boldsymbol{y} \in \{0,1\}^2} \widetilde{\mathrm{mul}}_2(g, \boldsymbol{x}, \boldsymbol{y})\, \widetilde{V}_2(\boldsymbol{x}) \widetilde{V}_2(\boldsymbol{y})$$

と書く。toy では

$$\widetilde{V}_1(g) = 12 + 10g$$

であり、第一ラウンド partial-sum は

$$q_1(T) = (9 + 10g)T^2 + (13 + 7g)T + (12 + 5g)$$

だった。つまり Virgo sumcheck の主張は

$$「親層の値」 = 「selector で重みづけした子層値の hypercube 和」 \tag{52}$$

である。

7.3 どこが本質的に違うのか

(1) 平坦化 vs. 層構造

Aurora rowcheck は、R1CS の各行を

$$(x+1)\cdot 1 = u_1, \quad (x+2)\cdot 1 = u_2, \quad (x+3)\cdot 1 = u_3, \quad u_1 u_2 = m, \quad m u_3 = y$$

のように**同じ row index の世界**へ平坦化して並べる。rowcheck はその行ごとの積だけを見る。したがって rowcheck 単体からは「row 4 の出力が row 5 の入力である」という深さ方向の情報は見えない。その部分は lincheck と R1CS 化の段階に吸収されている [4]。

一方 Virgo sumcheck は、

$$V_2 \to V_1 \to V_0$$

という **層構造**を保ったまま、layer ごとに claim を次の layer の claim へ還元する。このため、Virgo は「どの親ゲートがどの子ゲートを読むか」という配線情報を selector 多項式の中に直接持っている [5, 6]。

(2) divisibility の証拠 vs. partial-sum のメッセージ

Aurora の prover が本質的に与えるものは

$$q_{\mathrm{row}}(X) = \frac{f_a f_b - f_c}{Z_{H_1}(X)}$$

という **商多項式**である。これは「差が row-domain 上で消える」ことの証拠である。

それに対し Virgo の prover が各ラウンドで与えるものは

$$f_i(T) = \sum_{\text{残りの Boolean 変数}} F(r_1, \dots, r_{i-1}, T, \dots)$$

という **partial-sum 多項式**である。これは「巨大な hypercube 和を乱数点で潰しながら追跡する」ためのメッセージであって、Aurora の商多項式とは役割が違う [6, 5]。

(3) rowcheck 単体は wiring を持たない

Aurora rowcheck の入力は

$$(f_a, f_b, f_c)$$

であり、rowcheck 自体は

$$f_a(\alpha) f_b(\alpha) = f_c(\alpha)$$

しか見ない。どの f_a, f_b, f_c が本当に Az, Bz, Cz から来たか、すなわち wiring や線形写像の整合性は lincheck の仕事である [4]。

これに対し Virgo の sumcheck では

$$\widetilde{\mathrm{add}}_i(g, \boldsymbol{x}, \boldsymbol{y}), \qquad \widetilde{\mathrm{mul}}_i(g, \boldsymbol{x}, \boldsymbol{y})$$

が式の中に入る。したがって Virgo の sumcheck は、値の整合性と配線の整合性を一つの式の中で同時に運んでいる。

■結論　この意味で、**Aurora の rowcheck 単体**と **Virgo の 1 layer 分 sumcheck** を比較するのは、少し粒度がずれている。役割対応としてより近いのは

Aurora の rowcheck + 対応する lincheck　vs.　Virgo の layer sumcheck

である。

7.4　Virgo の一層を Aurora 風に flatten すると何が見えるか

ここは理解の橋渡しとして面白い。

layer 1 の二つの multiplication gate を、gate index $g \in \{0, 1\}$ を持つ「二行の表」として見る。すると left child, right child, output の値は

$$L(0) = 3, \quad L(1) = 5, \qquad R(0) = 4, \quad R(1) = 1, \qquad O(0) = 12, \quad O(1) = 5$$

なので、それぞれを $\mathbb{F}_{17}$ 上で一次補間すると

$$L(g) = 3 + 2g, \qquad R(g) = 4 + 14g, \qquad O(g) = 12 + 10g. \tag{53}$$

すると

$$\begin{aligned} L(g)R(g) - O(g) &= (3 + 2g)(4 + 14g) - (12 + 10g) \\ &= 11g(g - 1) \end{aligned} \tag{54}$$

となる。これは $g = 0, 1$ で消える。したがって (54) は、$\{0, 1\}$ という row-domain 上では Aurora 的な rowcheck に非常によく似ている。

■しかし Virgo はこれをそのまま使わない　理由は、Virgo/GKR が本質的にやりたいことが

「ゲート index 上の多項式が row-domain で消える」

ことではなく、

「親層の claim を子層の二つの random-point evaluation に還元する」

ことだからである。したがって Virgo は (54) のような flat な residual を正面から証明するのでなく、(47) のような selector-sum 恒等式を sumcheck で処理する [5, 6]。

7.5 比較表

観点	Aurora の rowcheck	Virgo の sumcheck
主役の座標系	R1CS の row index	layered circuit の gate label / Boolean hypercube
基本式	$f_a f_b - f_c = Z_{H_1} q_{\mathrm{row}}$	$\widetilde{V}_i(g) = \sum \widetilde{\mathrm{add}}_i(\cdot) + \widetilde{\mathrm{mul}}_i(\cdot)$
何を圧縮するか	行ごとの積 $a_i b_i = c_i$	親層の主張を子層の和へ
配線情報	rowcheck 単体には入らず、lincheck 側にある	selector 多項式の中に直接入る
prover の代表メッセージ	商多項式 q_{row}	partial-sum 多項式 $f_i(T)$
層の扱い	平坦化して失う	保持したまま再帰的に処理

8 全体の見取り図

ここまでの toy を、一文ずつで言い直すと次のようになる。

■Ligero　witness を表に並べ、表の整合性をランダムな行線形結合と少数列の opening で spot-check する。主役は「表」である [1]。

■zk-STARK　計算を実行トレースとして書き、隣接行の関係を AIR 制約にし、列多項式と合成多項式の低次数性を FRI で検査する。主役は「実行ログ」である [3, 2]。

■Aurora　回路を R1CS に平坦化し、z, Az, Bz, Cz を作り、lincheck で線形写像、rowcheck で Hadamard 積を検査する。主役は「R1CS とその三本ベクトル」である [4]。

■Virgo　回路を層ごとに保ち、$\widetilde{V}_i$ への claim を sumcheck で $\widetilde{V}_{i+1}$ への claim へ還元し、最後の random-point evaluation を zkVPD で開く。主役は「各層の値とその multilinear extension」である [5, 6]。

9 結論

本稿の要点は、次の三つに尽きる。

1. 同じ toy computation でも、Ligero, zk-STARK, Aurora, Virgo はそれぞれ **表、実行トレース、R1CS、layered circuit** として計算を眺める。
2. Aurora の rowcheck は

$$a \circ b = c \quad \Longleftrightarrow \quad Z_{H_1} \mid (f_a f_b - f_c)$$

という **flat な divisibility test** である。
3. Virgo の sumcheck は

$$\widetilde{V}_i(g) = \sum \big(\text{wiring selector}\big) \cdot \big(\text{child values}\big)$$

という **layered な hypercube-sum identity** を、random-point claim へ再帰的に還元する仕組みである。

したがって、**Aurora は「平坦化して検査する」系統、Virgo は「層を保って還元する」系統**と覚えると、かなり見通しがよくなる。

また、式レベルの対応としては、Virgo の一層を flat に潰すと

$$L(g)R(g) - O(g) = 11g(g-1)$$

のような Aurora 的 residual が現れる一方で、Virgo 本体はそれを直接証明するのでなく、selector-sum 恒等式を sumcheck する。この点が、両者の「似ているが同じではない」感触の正体である。

参考文献

[1] Ames, Hazay, Ishai, and Venkitasubramaniam, *Ligero: Lightweight Sublinear Arguments Without a Trusted Setup*, ACM CCS 2017. https://acmccs.github.io/papers/p2087-amesA.pdf

[2] Ben-Sasson, Bentov, Horesh, and Riabzev, *Fast Reed–Solomon Interactive Oracle Proofs of Proximity*, ICALP 2018. https://drops.dagstuhl.de/storage/00lipics/lipics-vol107-icalp2018/LIPIcs.ICALP.2018.14/LIPIcs.ICALP.2018.14.pdf

[3] Ben-Sasson, Bentov, Horesh, and Riabzev, *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*, IACR ePrint 2018/046. https://eprint.iacr.org/2018/046

[4] Ben-Sasson, Chiesa, Spooner, and Ward, *Aurora: Transparent Succinct Arguments for R1CS*, IACR ePrint 2018/828. https://eprint.iacr.org/2018/828

[5] Zhang, Xie, Song, Chen, and Wang, *Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof Systems*, IACR ePrint 2019/1482. https://eprint.iacr.org/2019/1482

[6] Goldwasser, Kalai, and Rothblum, *Delegating Computation: Interactive Proofs for Muggles*, STOC 2008. https://eccc.weizmann.ac.il/report/2007/128/